Evaluating Transfer Learning for Cross-Domain Object Detection and Classification in Pokémon Snap

Joseph Slater
Georgia Institute of Technology
Atlanta, GA 30332 USA
jslater33@gatech.edu

Hongbo Yan
Georgia Institute of Technology
Atlanta, GA 30332 USA
hyan326@gatech.edu

Noah Branch
Georgia Institute of Technology
Atlanta, GA 30332 USA
nbranch30@gatech.edu

Ian Crane
Georgia Institute of Technology
Atlanta, GA 30332 USA
icrane6@gatech.edu

Abstract

Our project evaluates whether models trained only on synthetic 2D Pokémon sprite images can generalize to real 3D Pokémon Snap gameplay frames. We generated 10,000 synthetic training images and evaluated on 165 manually labeled real game frames from the Beach level, containing 160 labeled Pokémon instances. We compared YOLOv8n, YOLO11n, and two stage YOLO + DINOv3 pipelines. Although the models learned the synthetic sprite domain well, they transferred poorly to real gameplay frames. The best end-to-end system, YOLO11n + DINOv3, correctly localized and classified only 12 of 160 real Pokémon instances. Error decomposition showed that detection was the main bottleneck: YOLO11n localized only 19 of 160 objects, while DINOv3 correctly classified 12 of those 19 detected crops. These results show that clean 2D synthetic training data is not sufficient for reliable detection in low resolution 3D game frames, and that future work should focus on reducing the synthetic to real localization gap.

1. Introduction/Background/Motivation

Our project aims to answer a simple question: If a model has only practiced on simpler 2D Pokémon images, can it still find Pokémon in real Pokémon Snap game screenshots? Given a game screenshot, we want the system to indicate where each Pokémon appears in the scene and determine what type of Pokémon it is. This task is not easy because the Pokémon in the game may appear at different positions, with different sizes, angles, and sometimes be obscured by the background, other objects, or other Pokémon.

The main difficulty of this project is that we do not have enough annotated real game screenshots. If we want to directly use real Pokémon Snap images to train the model, we need to first collect a large number of game screenshots, then manually draw frames for each Pokémon and write the correct category. This process is very slow and difficult to scale up. Therefore, we chose a cheaper method: using 2D Pokémon sprites to generate a large number of training images and automatically generating corresponding positions and category labels. These synthesized images are easier to produce and label, but they don't look the same as the real 3D game scenes. Our goal is to test whether a model trained on simple 2D images can transfer to real 3D game scenarios.

From the perspective of deep learning, this task is best modeled using the problems of object detection and classification. The model not only needs to identify which Pokémon are present, but also predict their specific positions in the image. Based on this structure, we compared YOLO detectors with a two stage pipeline. In this two stage process, YOLO first locates candidate Pokémon regions, and then DINOv3 classifies the cropped candidate objects. This design enables us to study the positioning error and classification error when transferring from synthetic 2D training images to real game frames.

This approach also gives this project a more general significance. Many computer vision tasks encounter similar data dilemmas: labeled data in the real world is expensive and difficult to obtain, while synthetic data, simulated data, or simplified images are easier to acquire. Synthetic data has been widely used in the training of computer vision models, especially in scenarios where real data is scarce or difficult to label [7]. In the research of transferring from simulation to real environments, domain randomization ran-

domizes the visual conditions in the synthetic environment, making the real world potentially regarded by the model as a variation within the training distribution, thereby alleviating the gap between simulated images and real images [12]. This makes our setting relevant beyond Pokémon Snap.

However, the use of synthetic data also brings about the core challenge of this project: domain shift. Although the synthetic training images and the real game frames share the same set of Pokémon categories, their visual appearances differ significantly. The synthetic images are based on clean 2D sprites, while the real game frames contain low-resolution quality, complex backgrounds, perspective changes, pose variations, motion blur, and partial occlusions. Prior domain adaptation research shows that differences between training and test distributions can significantly affect model generalization [3]. In our task, this distribution difference is particularly evident because the model is trained with 2D synthetic sprites and needs to recognize objects in a 3D game environment during testing. Thus, even if a model achieves high performance on the synthetic validation set, it may not maintain stable detection and classification capabilities in real Pokémon Snap game frames.

Due to the limited availability of real game frame annotations, we also rely on transfer learning, which uses knowledge learned from a source task or dataset to help a target task with less labeled data [11]. In our pipeline, YOLO is trained for Pokémon localization, while DINOv3 is used as a pretrained visual representation model for classifying cropped Pokémon targets [10].

Our project utilized two main datasets. The training set is a synthetic dataset consisting of 10,000 images, generated by automatically creating 2D Pokémon sprites and background scenes similar to those in the game. The categories were selected from the Pokémon encountered in the first level of Pokémon Snap. During generation, each Pokémon had a 15% chance of appearing in the image, and each image had a 25% chance of undergoing color saturation enhancement. The test set consists of 165 manually labeled real Pokémon Snap frames, containing 160 ground-truth Pokémon bounding boxes across 12 classes.

Therefore, our main research question is whether existing deep learning models can transfer knowledge from 2D synthetic data to real 3D game scenes, and where the main failures occur when this transfer does not work well.

2. Approach

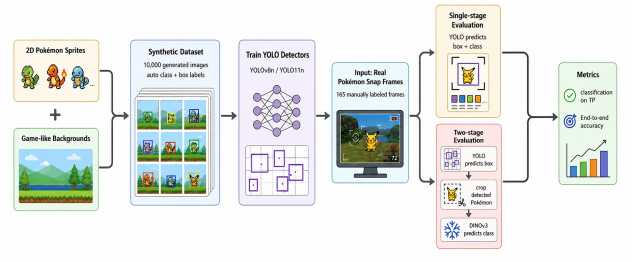

Figure 1. The whole pipeline.

We design a controlled synthetic to real evaluation pipeline for studying whether models trained on 2D Pokémon sprites can generalize to real 3D Pokémon Snap frames. The pipeline, shown in Figure 1, consists of several main stages, synthetic data generation, detector training, crop based classification, and cross domain error decomposition. The novelty of our approach is a controlled evaluation framework.

Given a gameplay frame, the system must output Pokémon instances, where each instance contains a bounding box and a species label. We therefore evaluate both one stage and two stage systems. The one stage baselines are YOLOv8n [5] and YOLO11n [6], which directly predict boxes and species labels. The two stage system uses YOLO to localize candidate Pokémon regions and DINOv3 [10] to classify the cropped regions. This design lets us measure localization and classification errors separately.

Because manually annotated real Pokémon Snap frames are expensive to collect, we generated 10,000 synthetic training images from 2D Pokémon sprites and game like backgrounds. The classes were selected from the Pokémon appearing in the first level. Each Pokémon had a 15% chance of appearing in a generated image, and each image had a 25% chance of color saturation augmentation. The sprite sources included Hugging Face [2], Kaggle [8], and PokeAPI [4]. We cleaned these sources by removing non 2D Pokémon images. For each inserted Pokémon, the generator automatically produced a class label and bounding box in YOLO format. The resulting class distribution and number of sprites per image are shown in Figure 2 and Figure 3.

To train the crop classifier, we converted the synthetic detection dataset into a classification dataset by cropping each ground-truth box with 10% padding. We used DINOv3, replaced its original head with a learned 12 class Pokémon classifier, and fine-tuned it on these synthetic ground truth crops using cross-entropy loss, AdamW, automatic mixed precision, and validation accuracy for checkpoint selection.

A central experimental focus of this project is analyzing

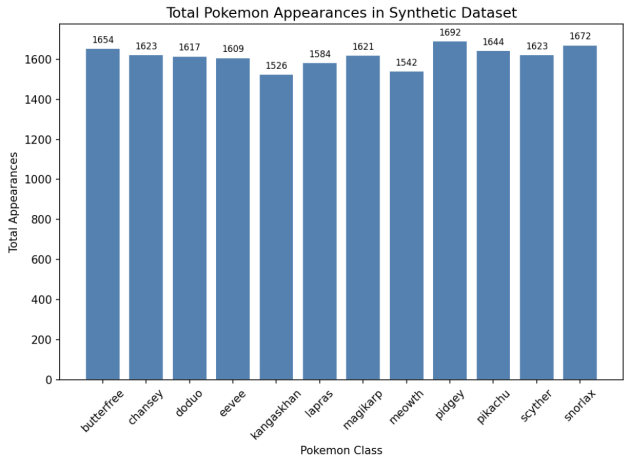

Figure 2. Class distribution in dataset.

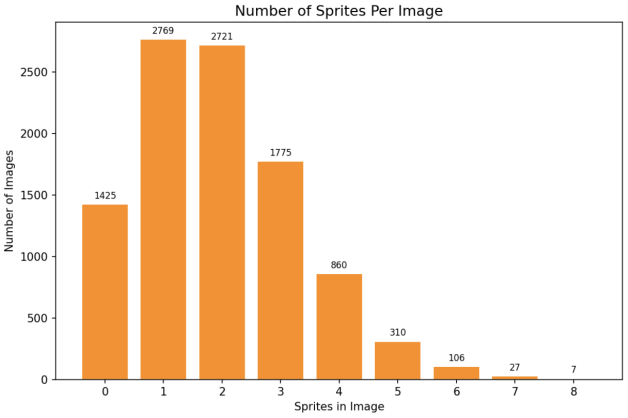

Figure 3. Frequency of sprites per image.

how the source domain of training data affects cross-domain generalization. We trained separate model variants on official 2D sprites featured on various backgrounds, then quantitatively compared their detection and classification performance on held-out frames extracted from Level 1 of the original Pokémon Snap.

The two stage pipeline is designed to determine whether cross-domain failure comes from localization, classification, or both. For each real test frame, YOLO predictions are sorted by confidence and greedily matched to unused ground truth boxes. Unmatched predictions are false positives, and unmatched ground truth boxes are false negatives. This class agnostic matching separates localization from species classification.

After matching, each YOLO box is cropped with padding and passed to the DINOv3 classifier. We report detector localization performance, classification accuracy on true positive detections, and full end-to-end accuracy. This evaluation design directly tests our main hypothesis: synthetic 2D sprite data may be sufficient for learning the training domain, but may not provide enough visual variation to generalize to low resolution 3D gameplay frames.

The main problem we anticipated was the domain gap between clean 2D sprites and real 3D gameplay frames, which include low resolution, perspective changes, motion blur, and cluttered backgrounds. This problem did occur in practice. Our first attempt was to train YOLO directly on the synthetic images and evaluate it on real frames. Although this worked well on the synthetic validation set, it transferred poorly to real frames, with many missed detections and unstable class predictions across adjacent video frames. This failure motivated the two stage evaluation, which allowed us to test whether the main bottleneck was localization or classification.

3. Experimental Setup and Results

3.1. Model Architecture

3.1.1 YOLOv8

The novel elements of the YOLOv8 object detection and classification model rely on a convolution neural network that utilizes a down-sampling block that helps in reducing the resolution of the image allowing the model to learn important features across the dataset. Darknet [9] is often used as a dense connection layer of the model that typically assists with carrying important features through the system. Next, a spatial pyramid pooling layer is utilized to give the model flexibility in its ability to handle varying image data without relying on data augmentation to create a robust model. [1] Finally, the model leverages an anchor-free approach that has simplified the task of regression for creating bounding boxes, where the detection head is decoupled from classification, so these two tasks are distinguished. [1]

3.1.2 YOLOv11

YOLOv11 improved upon the architecture of YOLOv8 by incorporating a C3K2 block [6] that acts as a bottleneck in the flow, where features are split up to reduce and focus the features being propagated through the model while also reducing the size of the data flowing through the system. [6]This helps improve the model's ability to quickly and accurately detect objects in the data. The model also features an improved spatial pyramid pooling feature that works to aggregate information across different aspect ratios and other variance in image data [6]. These changes have been documented to improve the model's ability to capture objects of varying sizes.

3.1.3 YOLOv11 x DINOv3

For this model, object detection and classification are being accomplished in two stages. Using YOLOv11 as the object detection model and the DINOv3 Vision Transformer [10] model was found to work well. The DINOv3 model is a self-supervised Vision Transformer that uses self-attention for visual representation learning. Other vision transformer architectures were evaluated, but DINOv3 was the most effective for this task and thus its architecture is the focus.

3.2. Loss Functions

For YOLO models, there are three loss functions that are used. The first is classification loss and the models employ Binary Cross-Entropy (BCE) loss or log loss with the following function:

$$\mathcal{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right] \quad (1)$$

BCE can be used for this type of multi-class classification since it can treat the different classes as independent occurrences and treat correct versus incorrect as a binary decision by using the sigmoid activation function on the outputs. In YOLO style detectors, classification loss is separate from box localization loss. The next type of loss seen in the YOLO models is the box loss or localization loss, which is the method associated with making sure that the bounding box around the detected object is accurate. It uses the Complete Intersection over Union, which can be seen in equation 2, and the definition of the variables in the function are defined in equation 3, 4, and 5, of the image to ensure that the shape of the box converges appropriately around the target image.

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (2)$$

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2 \quad (3)$$

$$\alpha = \frac{v}{(1-IoU)+v} \quad (4)$$

$$IoU = \frac{|A \cap B|}{|A \cup B|} \quad (5)$$

This is a standard method for evaluating how well an object detection model can generate a bounding box around an object using the predicted versus ground truth then nudging the bounding box towards the correct location. The last loss function associated with YOLO models is the distribution focal loss and is a special type of focal loss that helps to improve the accuracy of the box via optimization of the surrounding probability. This focal loss is used for the regression task of locating the object in the frame and as the predicted bounding box regresses towards the ground truth bounding box, the loss improves. [6]

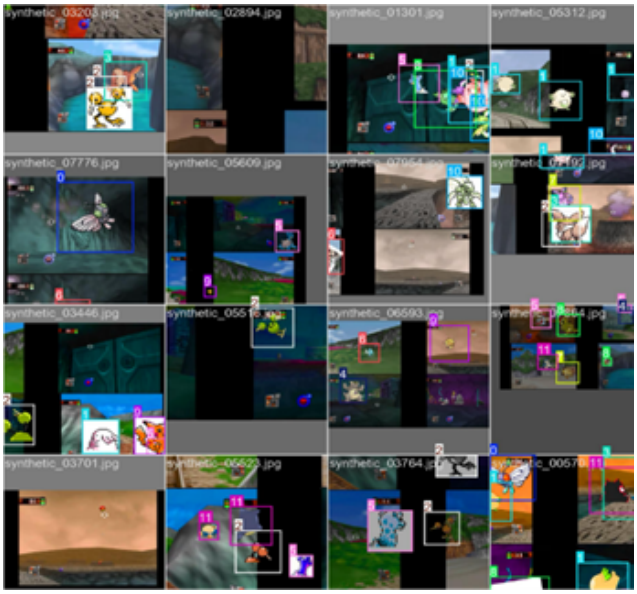

Figure 4. Training batches.

3.3. Training Scheme

For the training of the models, each team member was assigned a model to train and optimize. The hyperparameters were tuned initially by the Ultralytics [5] YOLO package where it determined the best optimizer, learning rate, and momentum automatically. This is a built-in function to the package itself and allows for the user to focus more on the dataset, or other nuanced parameters that could be used in later iterations of the model training phase. For the models in this paper, we opted for the automatic hyperparameter optimization and focused more on the incoming dataset as can be seen in the approach. The Ultralytics [5] package also generates training batch images to show what the model is being fed during training, which can be seen in Figure 4. This insight gives us a clue as to what the model is seeing during the training and helps the practitioner to optimize the dataset for the task at hand. There are lots of images of various Pokémon in the frame and shows that the system could capture lots of different objects during the detection phase. The images also show some of the data augmentation that was utilized to challenge the model and prevent it from only learning color or texture features during the training phase.

The main training and inference hyperparameters are summarized in the supplementary material in Table 4.

3.4. Testing Scheme

For the testing phase of the model, Roboflow was used to develop a 165-frame dataset that consisted of labeled im-

Class Name	Count
Butterfree	49
Chansey	5
Doduo	10
Eevee	8
Kangaskhan	6
Lapras	5
Magikarp	4
Meowth	20
Pidgey	23
Pikachu	10
Scyther	7
Snorlax	13
Total	**160**

Table 1. Class distribution of the test dataset.

ages that could be used to evaluate the inference capability of the models. Each image in the test dataset has an associated label that tells the model what class should be present in the frame and where it should be in there. The test dataset and the first level of the game contain some class imbalance, which can be seen in table 1. This is due to the appearance of each Pokémon that was built into the game and is not something that can be controlled for. However, it is a good test for the model since a computer vision model should be able to perform on all the objects that it is trained to detect, even if they tend to appear less frequently in the frames.

3.5. Results

We evaluate performance from three perspectives: synthetic-domain learning, transfer to real Pokémon Snap frames, and classification after detection.

Table 2 summarizes the main real-frame results. Tables 5 and 6 show that many failures were missed detections, while Tables 7 and 8 show that DINOv3 improved classification only after YOLO successfully detected a crop. Overall, the models learned the synthetic 2D sprite domain well, but this performance did not transfer reliably to the real game video, as shown in Figures 8–12.

As seen in the images for the varying YOLO models in Figure 5, 6 and 7, they consistently capture Pikachu at the beginning of the test video with higher confidence, but could not give a solid label to other Pokémon such as Butterfree or Doduo. Pikachu was recognized quite well, given the dataset contained a decent amount of Pikachu in varying positions. The Pokémon API specifically had a good amount of Pikachu with better resolution. The models often detected Pikachu with higher confidence near the beginning of the video, but struggled with other Pokémon such as Butterfree and Doduo. The larger models failed more so to recognize that a Pokémon was present let alone categorize anything.

An example would be Snorlax represented in Figure 12. The smaller models pick up on Snorlax consistently, however the larger models failed to see any Pokémon in that area. This suggests possible over specialization to the synthetic sprite domain and poor generalization to real game frames.

Confidence scores also played a crucial role in getting the models to detect anything. If the confidence was set to 0.50 or higher before running the prediction script for the Ultralytics Implementation, not much would be detected at all for any Pokémon outside of the Pikachu in the beginning. To get the models to detect anything consistently, the confidence had to be set relatively low meaning that many Pokémon were classified as the wrong type of Pokémon within several consecutive frames. There were certain instances where the confidence was relatively high such as the Butterfree example but an accurate classification was not always represented.

In consecutive frames, the same Pokémon sometimes flickered between labels, such as Butterfree being predicted as Eevee in one frame and correctly as Butterfree in the next.

Some visually prominent examples, such as Pikachu and Snorlax, were easier for certain models, but this did not generalize across classes. Every other model produced had variable predictability in identifying certain Pokémon.

YOLO11n produced more true positive detections than YOLOv8n in our experiments, suggesting a modest improvement. However, the improvement was not large enough to overcome the synthetic to real frame gap. However, both models still had low recall on real frames.

We also evaluated two stage variants in which YOLO produced crops and a vision transformer classified them. Among these, YOLO11n + DINOv3 gave the strongest classification accuracy on true-positive detections.

The number of True Positive detections was 19 for YOLO11n versus 11 for YOLOv8n. This suggests that YOLO11n may provide better localization in this setting.

Overall, the experiment yielded three main conclusions. Firstly, the model successfully learned the knowledge in the synthetic Pokémon domain, as evidenced by its excellent validation performance. Secondly, this performance cannot be reliably transferred to real Pokémon snap frames, where the miss rates were high for most categories. Thirdly, the YOLO11n + DINOv3 pipeline improves the classification effect for the detected Pokémon, but the entire system is still limited by the recall rate of the detector. These findings support our conclusion that the main challenge lies in the transfer from the synthetic domain to the real domain, especially the gap between clean 2D images and 3D game frames.

To make this bottleneck explicit, we decompose the error in the two stage system. The two stage pipeline sepa-

Model	TP	FP	FN	Det. P	Det. R	Det. F1	Cls. Acc. on TP	Cls. Macro F1	End-to-end
YOLOv8n	11	11	149	0.500	0.069	0.121	36.4% (4/11)	0.172	2.5% (4/160)
YOLO11n	19	23	141	0.452	0.119	0.188	52.6% (10/19)	0.335	6.3% (10/160)
YOLOv8n + DINOv3	11	11	149	0.500	0.069	0.121	63.6% (7/11)	0.361	4.4% (7/160)
YOLO11n + DINOv3	19	23	141	0.452	0.119	0.188	**63.2% (12/19)**	**0.371**	**7.5% (12/160)**

Table 2. Overall comparison of model performance on real Pokémon Snap frames. TP, FP, FN, detection precision, detection recall, and detection F1 measure localization. *Cls. Acc. on TP* is the species-classification accuracy on YOLO's true-positive detections. *Cls. Macro F1* is the macro-averaged classification F1 over the 12 Pokémon classes, computed only on true-positive detections. *End-to-end* is the proportion of all ground-truth Pokémon that were both correctly localized and correctly classified.

rates localization errors from classification errors. For the best system, YOLO11n + DINOv3, the real test set contained 160 ground-truth Pokémon instances. YOLO11n localized only 19 of them, so detection recall was 19/160 = 11.9%. DINOv3 correctly classified 12 of these 19 detected crops, so classification accuracy on true-positive detections was 63.2%. The final end-to-end accuracy was 12/160 = 7.5%.

This shows that localization, not classification, was the main bottleneck. Since 141 of 160 Pokémon were missed before classification, even a perfect classifier could only reach 11.9% end-to-end accuracy with this detector. Future work should therefore prioritize reducing the synthetic-to-real gap for detection.

4. Conclusion and Future Work

Our project studied whether training on synthetic 2D Pokémon sprites is sufficient for detecting and classifying Pokémon in real 3D Pokémon Snap gameplay frames. The models learned the synthetic sprite domain well, but they did not transfer reliably to real game frames. All models were trained on synthetically generated data formed by taking 2D sprites and testing them on the Pokémon snap backgrounds. Despite sprites being color shifted, size changed and rotated, results remained strong. Even in the case of sprites which are heavily obscured by the placement of another sprite, the models still did a good job detecting them. These positive results did not translate to the 3D space. On real 3D game frames, all models had low detection recall and failed to consistently detect Pokémon. These findings directly show the difficulty of 2D to 3D generalization.

It is worth noting how larger more complex models fared worse than smaller models. The more complex models specialize in sprites, a 2D space, and were unable to generalize to the 3D space of Pokémon Snap. Even with precautions such as dropout layers being introduced, more complex models could not compete with smaller models at this task. With these smaller models, some Pokémon fared much better than others. Pokémon which took up significant portions of the frame, such as Pikachu and Snorlax were often classified correctly. One possible reason could be due to these specific Pokémon having sprites which better represent their poses and postures in Pokémon Snap. There is also the possibility that by taking up more of the frame, the model can focus on more nuanced features when attempting classification.

The most noteworthy fact was the best performing model. While no models did particularly well, the custom two stage model, built with YOLO11n as the detector and DINOv3 as the classifier did perform the best. In cases where YOLO11n detected a Pokémon, DINOv3 successfully classified it 63.2% of the time. However, the full end-to-end system correctly localized and classified only 12 of 160 ground-truth Pokémon instances, or 7.5%. This suggests that separating detection and classification can help diagnose the failure, but it does not solve the main localization bottleneck. This separation allows us to measure detection and classification errors independently.

There are several avenues which one can take to advance this research. One could continue testing these two model architectures to bridge the gap of extreme domain shifts, perhaps by selecting a different detector or classifier. Another option would be for one to introduce temporal consistency. While our empirical results were gathered from frame data, the true input is in fact game footage, so there is the possibility that using features captured in a temporal setting (such as a Butterfree's wings flapping) could help with both detection and classification. Both proposals would further the research conducted in this paper.

Overall, this work demonstrates that the 2D to 3D domain gap is significant. Even after creating synthetic data to best mimic the 3D space, single stage models still failed to meaningfully capture the 3D space. However, the two stage architecture of YOLO11n and DINOv3 still achieved only 7.5% end-to-end accuracy, even though it reached 63.2% classification accuracy on detected Pokémon. These findings suggest similar risks in settings where models are trained on clean synthetic images and tested on visually different real images.

5. Work Division

See table 3.

Student Name	Contributed Aspects	Details
Joseph Slater	Data Creation and Analysis	Scraped the sprites and implemented algorithms to generate synthetic dataset. Collected and labeled test set in Roboflow. Reviewed and analyzed all model results to write conclusion and future work section.
Hongbo Yan	Training and Analysis	Trained the YOLO11n + DINOv3 two stage pipeline, and completed abstract and introduction/motivation/background.
Noah Branch	Training and Analysis	Fine tuned different variants of YOLO models using ultralyics to compare them. Specifically focusing on variations of the YOLO8 model.
Ian Crane	Training and Analysis	Trained single-stage object detection and classification model (YOLOv11), tuned hyperparameters, and completed analysis of model results.

Table 3. Contributions of team members.

References

[1] A. Al-Karkhi. Understanding yolov8 architecture: A breakthrough in real-time artifact detection, 2023. 3

[2] fcakyon. pokemon-classification. https://huggingface.co/datasets/fcakyon/pokemon-classification, 2023. Accessed: May 5, 2026. 2

[3] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, Francois Laviolette, Mario Marchand, and Victor Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35, 2016. 2

[4] Paul Hallett and PokéAPI Contributors. PokéAPI: The RESTful Pokémon API. https://pokeapi.co/. Accessed: May 5, 2026. 2

[5] Chaurasia A. Qiu J. Jocher, G. Ultralytics yolov8, 2023. https://docs.ultralytics.com/models/yolov8/. 2, 4

[6] Rahima Khanam and Muhammad Hussain. Yolov11: An overview of the key architectural enhancements, 2024. arXiv:2410.17725. 2, 3, 4

[7] Sergey I. Nikolenko. Synthetic data for deep learning, 2019. arXiv:1909.11512. 1

[8] noodulz. 1000 Pokemon dataset. https://www.kaggle.com/datasets/noodulz/pokemon-dataset-1000, 2024. Accessed: May 5, 2026. 2

[9] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement, 2018. arXiv:1804.02767. 3

[10] Oriane Simeoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michael Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothee Darcet, Theo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Herve Jegou, Patrick Labatut, and Piotr Bojanowski. Dinov3, 2025. arXiv:2508.10104. 2, 4

[11] Chuanqi Tan, Fuchun Sun, Tao Kong, Wenchang Zhang, Chao Yang, and Chunfang Liu. A survey on deep transfer learning. In *Artificial Neural Networks and Machine Learning – ICANN 2018*, pages 270–279, 2018. 2

[12] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017. 2

6. Supplementary Material

6.1. Code

Our repository contains synthetic data processing scripts, YOLO training wrapper programs, DINOv3 classification training code, cross-domain evaluation code, and video inference visualization code. Our code has been uploaded to the attachment. Our GitHub address is https://github.com/TechsTeamRocket/Pokemon-Visualization-Tool

6.2. Figure

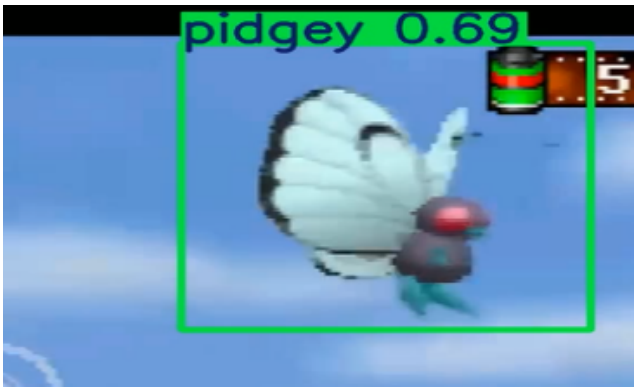

Figure 5. Butterfree incorrectly identified

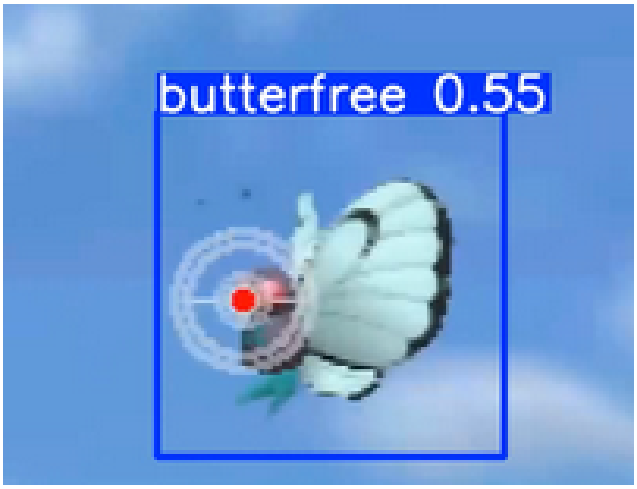

Figure 6. At 43 seconds into the test video, Butterfree is correctly identified in one frame and incorrectly identified in the next frame.

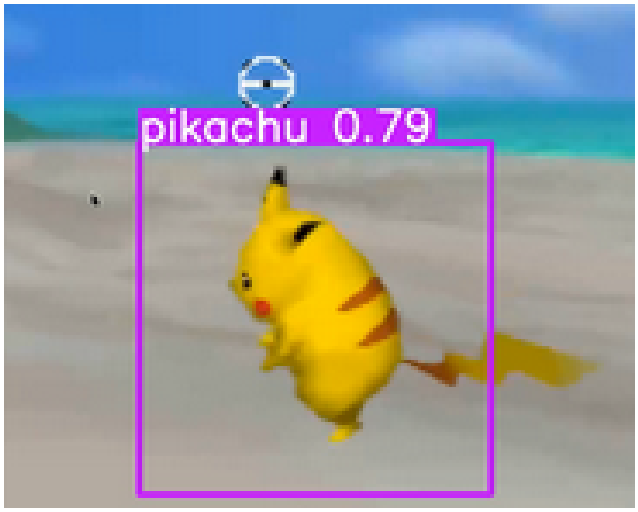

Figure 7. Pikachu correct classification

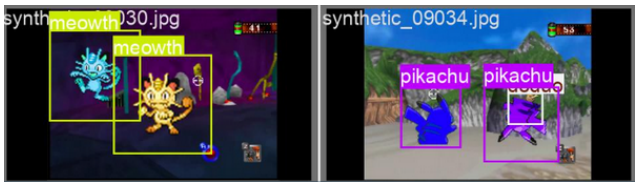

Figure 8. Validation results on YOLO8 finetuning process on validation set

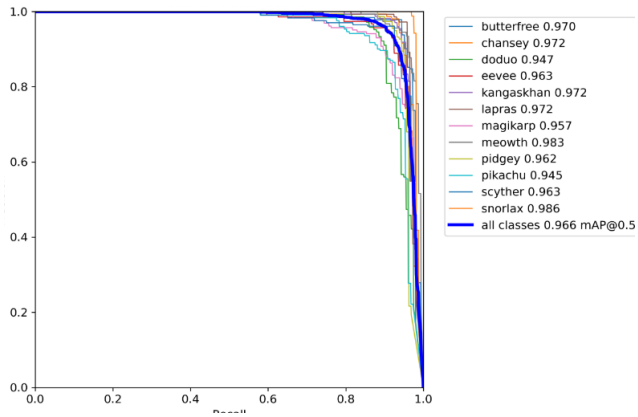

Figure 9. YOLO8 Recall curve

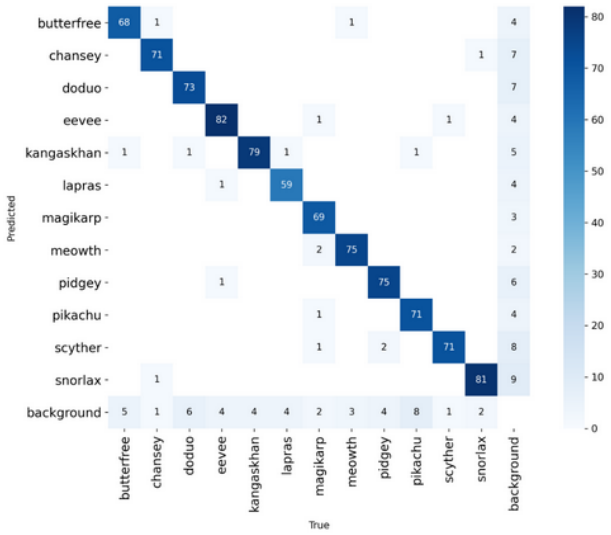

Figure 10. validation confusion matrix of YOLO8 model for training

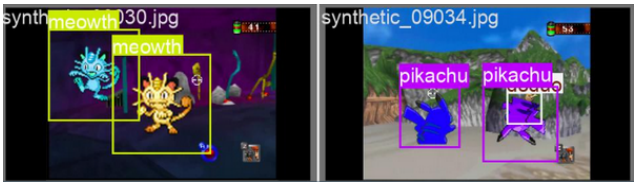

Figure 11. Validation results on YOLO8 finetuning process on validation set

Figure 12. YOLO8 nano on the top and YOLO8 large on the bottom at time stamp 57

6.3. Tables

Hyperparameter	YOLO Detector	DINOv3 Classifier
Initial weights	yolov8n.pt / yolo11n.pt	vit_base_patch16_dinov3
Input size	640×640	256×256
Batch size	16	64
Epochs	50	15
Optimizer	SGD	AdamW
Initial LR	0.01	1e-4
Momentum	0.937	—
Weight decay	5e-4	1e-4
AMP	true	true

Table 4. Training hyperparameters.

Class	Total	Missed	Miss Rate
Butterfree	49	43	87.8%
Chansey	5	2	40.0%
Doduo	10	8	80.0%
Eevee	8	8	100.0%
Kangaskhan	6	6	100.0%
Lapras	5	5	100.0%
Magikarp	4	3	75.0%
Meowth	20	16	80.0%
Pidgey	23	23	100.0%
Pikachu	10	9	90.0%
Scyther	7	7	100.0%
Snorlax	13	11	84.6%

Table 5. YOLO 11 miss rates

Class	Total	Missed	Miss Rate
Butterfree	49	44	89.8%
Chansey	5	4	80.0%
Doduo	10	8	80.0%
Eevee	8	8	100.0%
Kangaskhan	6	6	100.0%
Lapras	5	5	100.0%
Magikarp	4	4	100.0%
Meowth	20	20	100.0%
Pidgey	23	22	95.7%
Pikachu	10	10	100.0%
Scyther	7	7	100.0%
Snorlax	13	11	84.6%

Table 6. Miss rate results of YOLOv8 on the test dataset.

Class	YOLO11n Find Pokémon	DINOv3 Recognize Pokémon	DINOv3 classification accuracy
Butterfree	6	2	33.3%
Chansey	3	3	100%
Doduo	2	2	100%
Eevee	0	0	—
Kangaskhan	0	0	—
Lapras	0	0	—
Magikarp	1	0	0%
Meowth	4	2	50%
Pidgey	0	0	—
Pikachu	1	1	100%
Scyther	0	0	—
Snorlax	2	2	100%
TOTAL	19	12	63.2%

Table 7. YOLO11n+DINOv3 two stage pipeline results.

YOLO Model	**Vision Transformer**	**TP**	**FP**	**FN**	**YOLO TP Rate**	**ViT TP Rate**	**Δ**
YOLOv8n	AIMv2	11	11	149	0.3636	0.4545	+0.09
YOLOv8n	DINOv3	11	11	149	0.3636	0.6364	+0.27
YOLO11n	AIMv2	19	23	141	0.5263	0.4737	-0.05
YOLO11n	DINOv3	19	23	141	0.5263	0.6316	+0.11

Table 8. Comparison of YOLO detection results and Vision Transformer classification results on true positive detections.